Tell me about a project at risk of missing its deadline. What did you do?

Describe a time you had to re-scope a project mid-flight.

How do you handle competing priorities when multiple projects need attention?

How do you handle disagreements with a PM or another EM?

How do you create alignment between Engineering and Product when roadmaps conflict?

Tell me about a time you had to decide with incomplete data.

Describe a time you had to make an unpopular decision. How did you bring people along?

What was the hardest decision you made on that project?

Walk me through your most complex project from inception to launch.

How did you manage stakeholder communication when things went off track?

E2E Redesign

At Ancestry, I led the rebuild of our onboarding flow with a hard deadline tied to RootsTech — the single biggest public event in the genealogy world, drawing tens of thousands of users and tied directly to our acquisition targets for the year. When the final designs landed, we were looking at five months of work in a three-month window.

I ran a t-shirt sizing session with the team, added buffer tickets for unknowns, and came back with a concrete capacity model. I immediately facilitated a prioritization exercise with the PMs — sorting every feature into must-have, nice-to-have, and can-come-later. We aligned on a phased rollout. That conversation was only possible because I had numbers to anchor it, not opinions.

Mid-development, we hit a regression — hint generation rates weren't matching the legacy flow. Hints are the core hook of the Ancestry experience and one of our primary success metrics. I called an SOS with stakeholders, explained exactly what was broken and why it mattered, then paired directly with a senior engineer to dig into root cause. We found it — a change in how user data was being ingested. I came back to product with a specific trade-off: move one MVP feature of equivalent effort out to the follow-up release, pull the hint fix in. A decision, not a problem. They agreed, I re-sequenced the team, and we stayed on track.

The MVP launched on time at RootsTech. Hint generation rate increased 2% over legacy, build-through rate up 5%, onboarding time dropped from 60 to 40 minutes. The legacy flow was fully deprecated after the MLP two months later.

What I learned is that the estimation session didn't just give me numbers — it gave me credibility. Because I'd already shown product I worked from data, not gut feel, they trusted my trade-off proposal immediately when the crisis hit. The real lesson isn't about estimation — it's about

building decision-making credibility in the first month, so you have it when you actually need it in month two.

How do you balance technical debt vs. shipping velocity?

Tell me about a failure and what you learned.

What would you do differently if you ran it again?

What did the team learn from this project?

Describe a time a project failed. What was your role and what did you learn?

Tell me about a time there was a production failure

The Story: The "5% Edge Case" Cascading Failure

"Recently, we deployed a new feature to our production environment (L1). While our automated tests and manual smoke tests passed, we began experiencing a cascading failure about an hour after deployment. It turned out that a specific condition in a legacy flow—used by only 5% of our users—was causing a fatal exception that crashed the entire service instance. Because these instances were crashing and restarting constantly, it began impacting the 95% of healthy traffic, leading to a site-wide degradation.

As the EM on point, my immediate priority was three-fold: stop the bleeding (recover the site), coordinate the 'War Room' with my senior engineers, and perform a deep-root cause analysis (RCA) to ensure this type of 'cross-flow' contamination could never happen again.

I led the response through three distinct phases:

The 'Rollback First' Policy: As soon as the alert triggered, I pulled in the on-call engineer and the person responsible for the rollout. Within 5 minutes of identifying the instability, I made the call to trigger a full rollback. I have a strict policy: in production, we don't 'fix forward' while the site is down. We restore service first, then investigate.

The 'Micro-RCA': After the rollback, we dug into the logs and found a unique architectural flaw. The 5% flow was hosted within the same process as the primary flow. A memory-leak exception in the legacy code was killing the container, which took the healthy traffic down with it.

Systemic Hardening: I didn't want to just fix the bug; I wanted to fix the process. I implemented three major changes:

Alerting Thresholds: I lowered our error-rate thresholds to catch low-volume/high-severity anomalies that our previous 'average' metrics were smoothing over.

Mandatory Canary Windows: I implemented a new deployment rule where all code must sit in a Canary environment for 4 hours with specific 'Instance Health' monitors before promoted to 100%.

Architectural Isolation: I initiated a project to decouple these legacy flows into their own microservice so that a failure in an unmaintained flow could no longer impact our core business logic.

The impact was immediate. We restored the site to 100% health within minutes of the rollback. More importantly, the new Canary and Isolation strategy has since caught two other edge-case bugs before they ever hit a single 'real' user. The team now has much higher confidence in our deployment pipeline, and we've reduced our Mean Time to Recovery (MTTR) by having clearer ownership over isolated service flows.

How do you handle a situation where your team is blocked by another team?

Tell me about a time you aligned multiple teams with conflicting priorities.

Describe a time you had to influence a decision you had no authority over.

Describe a time you had to make an unpopular decision. How did you bring people along?

Walk me through your most complex project from inception to launch.

Paid hints story

At Ancestry I was leading an initiative that required integrating five separate engineering teams, each owning a different piece of the flow. None of them had us in their roadmap. Leadership wanted results in three months. On paper, we were blocked before we started.

My first instinct was not to escalate. Escalation would have created resentment — nobody wants their EM told by leadership to drop everything for another team's project. So I did something different. I went to each EM individually, one on one, before any group meeting happened. Not to ask for help — to understand their world. What were they working on? What was their team's capacity? What would make a request from me easy to say yes to versus hard?

Those conversations gave me two things. First, real information — I could see exactly where the genuine constraints were versus where there was quiet capacity. Second, relationships. By the time I brought everyone into a room together, I wasn't a stranger asking for favors. I was someone who understood their problems.

From there I did three things to systematically reduce the block. First, I mapped every dependency across all five teams into flow diagrams and shared them openly — nobody had seen the full picture before, and making the interdependencies visible created shared ownership of the problem rather than bilateral negotiations. Second, I built a shared integration component on my team's side that cut the implementation burden for other teams by roughly 50%. I made it easy to say yes by reducing what yes actually cost them. Third, I ran a two-week lightweight experiment first — before asking anyone to commit to the full build. When that came back positive, I walked into every conversation with data instead of a request. The dynamic shifted completely. I wasn't asking teams to prioritize my project on faith — the experiment had already proven the business case.

The full implementation wrapped in under a month after validation. The shared component cut testing and integration effort for all five teams by about half. And critically — none of those EMs felt steamrolled. Three of them became active collaborators on the follow-up work because the process had built genuine trust rather than burning it.

What I learned: being blocked by another team is almost never really about the other team. It's about whether you've made it easy enough for them to help you. The teams weren't blocking us — they were just full. My job was to reduce the cost of their yes until it became the obvious choice.

Planning and execution cycles

My planning philosophy is simple: the goal of any planning cycle is to surface unknowns as early as possible, because unknowns discovered in week one cost a fraction of what they cost in week eight.

In practice my process runs in three phases.

The first is estimation done right. I don't accept top-down timelines as starting points. I run bottom-up t-shirt sizing with the team — breaking requirements into tickets, explicitly accounting for hidden work like integration complexity, testing, and any dependencies on systems we don't fully own. I also require buffer tickets for any project spanning two or more months. Not because I expect things to go wrong, but because they always do, and the buffer is where unknowns live until they have names. This gives me a capacity model I can defend with data, not gut feel — which completely changes the nature of the conversation with product.

The second phase is ruthless tiering before a line of code is written. I sit with the PM and categorize every feature into must-have, nice-to-have, and can-come-later. The goal isn't to cut scope — it's to create a shared agreement on what done means at each milestone. On the onboarding rebuild at Ancestry, this saved us mid-project. When we hit a hint generation regression six weeks in, I already had product's trust because we'd built the plan together from data. I could walk in with a feature swap proposal and get a decision in one conversation instead of a week of negotiation.

The third phase is execution with visibility, not micromanagement. I establish a weekly heartbeat sync with stakeholders — not to report status but to surface blockers early. The distinction matters: status reporting is backward-looking, blocker surfacing is forward-looking. I track project health at the outcome level, not the task level. My signal that something is drifting isn't a missed ticket — it's engineers going quiet about uncertainty.

What's changed in my process over time is how I handle initiatives where the core hypothesis hasn't been validated yet. I used to plan those the same way I planned everything else — full estimation, full commitment, full build. Now I default to running a lightweight experiment first, in parallel with early infrastructure planning, before asking teams to commit to a full roadmap. On the paid hints initiative I had five teams and a three-month deadline. Instead of fighting for five months of roadmap across all five teams upfront, I ran a two-week experiment using Adobe Target to validate that users would actually pay for hints. When results came back positive, I walked into every cross-team conversation with data instead of a request. The project that looked politically impossible before the experiment became straightforward after it.

The underlying principle connecting all of it: planning isn't about predicting the future accurately. It's about building the organizational trust and shared context that lets you adapt fast when the future surprises you — which it always does.

Escalate vs. solve it yourself

My rule is this: I solve it myself when the decision is within my team's blast radius and speed matters more than consensus. I escalate when the decision requires authority, resources, or buy-in that I don't have — but I never escalate empty-handed.

The clearest example of not escalating was a production cascading failure. A legacy flow used by 5% of our users was crash-looping service instances and taking healthy traffic down with it. Site-wide degradation within an hour of deployment. I made the rollback call within five minutes of identifying the instability. No escalation, no committee. My policy in production is restore service first, investigate second — and that decision was entirely within my authority to make. Waiting for sign-off would have cost us time we didn't have.

The escalation happened after — but it wasn't about the incident itself. It was about the systemic fixes. Decoupling legacy flows into isolated workers, implementing mandatory canary windows — that work displaced two features we'd planned that quarter. That's a business tradeoff that isn't mine to make unilaterally. So I brought the incident data to product and made the case: the blast radius of another cascading failure outweighs the opportunity cost of deferring two features. They agreed immediately because I wasn't asking them to take my word for it — I was asking them to look at the data.

The paid hints initiative is the other side of that coin. I was trying to get five engineering teams to prioritize work that wasn't in their roadmaps. I could have escalated to leadership immediately and asked them to mandate it. But that would have created resentment — nobody wants their EM told by leadership to drop everything for someone else's project. So I went to each EM individually first, built the relationships, reduced the cost of their yes by building a shared integration component, and ran a two-week experiment to generate the data that would make the business case for them. Only then did I bring leadership in — not to force the decision, but to formalize the alignment that already existed.

The failure mode I watch for in myself is escalating too late because I'm trying to solve something that was never mine to solve alone. The experiment framework on paid hints was partly a check on that instinct — by validating the hypothesis first, I gave leadership a clear decision point rather than a vague request for cross-team resources.

The underlying principle: escalation isn't a last resort and it isn't a first move. It's the right tool when the decision requires something you don't have. The judgment is knowing exactly what you have and what you don't — and being honest about that line before the situation forces it.

Time when i escalated too late

The honest reframe: the legacy flow that caused the cascading failure had been sitting in the codebase for a long time. The architectural risk — that it shared process space with healthy traffic — was knowable before the incident. You could have escalated it as a blast-radius risk during planning, before it became a production incident.

The answer: "In retrospect, the architectural debt that caused the cascading failure wasn't invisible — it was just deprioritized. I knew that legacy flow was unmaintained. What I didn't do was frame it explicitly as a blast-radius risk to leadership and get it on the roadmap before it forced its way there through an incident. I escalated fast when the site went down, but I should have escalated the risk weeks earlier when it was still cheap to fix. That's directly why I now require explicit risk tickets for any legacy system that shares process space with production traffic."

The honest reframe: when leadership set the three-month deadline for six unfamiliar open-source stacks, you went through the full estimation process before surfacing the problem. That process took time. During that window, the team was already being mentally committed to an impossible timeline by leadership. Escalating the risk signal earlier — even before full estimation was complete — would have given leadership more time to adjust expectations and involve the other teams sooner.

The answer: "On Project Arborist, leadership set a three-month deadline before I'd had a chance to fully assess the complexity. My instinct was to do the work properly — complete the estimation, map all six dependencies, build the full picture — before bringing concerns to leadership. That took about two weeks. In hindsight, I should have escalated a preliminary risk signal much earlier — something like 'I haven't completed estimation yet but the surface area here is significantly larger than what the deadline assumes — we should talk before commitments solidify.' Instead I waited until I had the complete picture, by which time some stakeholders had already mentally committed to the original timeline, which made the conversation harder than it needed to be. Now I escalate a risk signal as soon as I sense it, even before I have all the data — with the explicit framing that it's preliminary."

Describe a time you had to re-scope a project mid-flight.

About six weeks into our onboarding rebuild at Ancestry, we hit a regression we hadn't anticipated. Hint generation rates weren't matching the legacy flow — and hints are the core hook of the Ancestry experience. They're what drives users deeper into the product. This wasn't a nice-to-have metric. It was one of our primary success measures for the launch.

I had two options. I could flag it as a known risk, ship anyway, and hope the regression wasn't as bad in production as it looked in testing. Or I could treat it as a launch blocker and find a way to fix it without blowing the RootsTech deadline.

I chose the second — but the decision wasn't just technical. The harder part was how to bring product along without triggering a panic or a blame conversation.

I did three things in sequence. First, before any stakeholder conversation, I paired directly with a senior engineer to fully understand the root cause. I didn't want to walk into a room with a problem — I wanted to walk in with a diagnosis. We found it: a fundamental change in how user data was being ingested was driving the discrepancy. Second, I quantified the fix in terms product could reason about — not in engineering hours, but in feature equivalence. The fix was roughly equivalent in effort to one MVP feature we had in scope. That framing was deliberate. It turned a scary conversation about regression and delay into a concrete trade-off decision. Third, I called an SOS with stakeholders and led with the business impact first, not the technical details. Here's what's broken, here's why it matters to our launch metrics, here's what I propose — move this one feature to the follow-up release, pull the hint fix into MVP scope. A decision, not a problem.

Product agreed in that same meeting. I re-sequenced the team's work that afternoon. We stayed on track for RootsTech.

The MVP launched on time at 50% traffic. Hint generation rate came in 2% above the legacy flow — reversing the regression we'd feared. Build-through rate improved 5%, onboarding time dropped from 60 to 40 minutes.

What I learned about re-scoping mid-flight is that the technical decision is usually the easy part. The hard part is the communication architecture around it. Three things made this work: I came with a diagnosis not just a problem, I framed the trade-off in product's language not engineering's language, and I moved fast — the longer a known problem sits unaddressed the more it erodes trust. Early, specific, and solution-forward is the only way to re-scope without breaking the relationship

What would you have done if product had pushed back on the feature swap?

I had a ranked list of alternatives ready before I walked in. The feature swap was my preferred option because the effort equivalence made it the cleanest trade-off. But if they'd pushed back I would have proposed a hybrid — shipping the MVP with a degraded hint experience and a hard commitment to fix it in a hotfix within two weeks of launch, with explicit stakeholder sign-off on the degraded metric. The one thing I wouldn't have done is ship without telling them.

How do you track project health without micromanaging?

My philosophy is simple — I don't track tickets, I track progress. Tickets tell you what people are doing. Progress tells you whether you're going to get where you need to go. Those are very different things, and confusing them is how EMs accidentally become micromanagers.

in our DSU, the goal is not to identify how many points or tickets were completed, the goal is to find blockers

In practice, progress means three things to me.

The first is metric movement. Before any project starts I work with product to define what success looks like at each milestone — not just delivery dates but the actual numbers that tell us the product is working. On the onboarding rebuild, that meant hint generation rate, build-through rate, and onboarding completion time. When the hint regression surfaced six weeks in, I caught it early because I was watching the right metric. No ticket told me something was wrong — the number did.

The second is blocker velocity. I restructured our daily standup around a single principle — the goal isn't to report how many points or tickets were completed yesterday. The goal is to find blockers. That one change transformed the DSU from a status theatre exercise into an actual problem-solving tool. Engineers stopped reciting task lists and started surfacing the things that were slowing them down. And once blockers are visible, they get resolved — which is the only thing that actually moves progress.

The third is the human signal, which no dashboard captures. On one project, sprint velocity dropped over two consecutive sprints. I didn't pull up the burndown chart — I went into 1:1s. What I found was that a senior engineer's PR review style was creating hesitation across the team. Engineers were avoiding opening PRs. The velocity drop wasn't a planning problem, it was a psychological safety problem. A ticket tracker would never have surfaced that. The only way I found it was by watching how the team was behaving, not what they were completing.

The signal I watch most closely is whether engineers are surfacing uncertainty. A team actively flagging risks is a healthy team. A team that goes quiet is almost always stuck but doesn't feel safe saying so. That silence is the most important health indicator I have — and you only hear it if you've built enough trust that raising a flag doesn't feel risky.

So when you ask how I track project health without micromanaging — the answer is I've replaced ticket monitoring with three things: metric movement, blocker velocity, and human signal. Together they give me earlier warning than any task tracker, without making engineers feel watched.

Tell me about a time you had to deliver with an incomplete or under-resourced team

Last year at Ancestry, our entire product and UX org was laid off overnight during a strategic pivot. My engineering team of twelve went from daily collaboration with product managers and designers to complete silence. No requirements, no roadmap, no design direction — for a full month before a new product team was assigned. And then another month while that team ramped up and developed a plan.

The constraint wasn't technical. It was organizational. I was trying to keep twelve engineers productive, engaged, and retained through what felt like freefall.

I did four things.

First, I ran one-on-ones with every engineer within 48 hours of the layoff. Not to give them answers I didn't have, but to understand where each person was. Some were updating resumes. Some were disengaged in meetings. Some were genuinely fine but worried about their colleagues. I needed to know which situation I was actually managing before I could do anything useful.

Second, I gave them the honest picture — not reassurance, honest information. Engineering wasn't impacted. Our skills weren't obsolete. The product function was cut because it was being restructured, not because the technical work had lost value. I didn't overpromise job security — I gave them the factual basis for why our team was safe and let them draw their own conclusions. People can handle uncertainty. What they can't handle is feeling like they're being managed rather than trusted.

Third, I made the decision not to let the team sit idle waiting for direction. I kept the team moving through our existing sprint backlog under the assumption the new product team would evaluate what aligned with their vision. Simultaneously I directed the team toward AI capability building — learning that would be valuable regardless of which direction the new product team took us. This kept everyone productive without committing to anything that might get thrown away.

Fourth — and this is the part that paid off most — I started meeting individually with the incoming PMs as soon as they were assigned, before they had finalized anything. I identified early that tree-building functionality was central to their vision and made the deliberate call to start building toward it without final specs. I told my team directly: some of this work might get thrown away. I wanted them bought in, not blindsided. When final requirements landed a month later, we already had a working solution close to target. Build time dropped from an estimated four months to three. Conversion improved 10%, NPS improved 15%.

Zero attrition through the entire period. Not one engineer left.

What I learned is that under-resourcing is almost always a leadership test before it's a delivery test. The engineers were watching how I handled the uncertainty more than they were watching what work I assigned them. The moment I showed up with honest information, a clear decision about how to use their time, and genuine trust in their ability to handle ambiguity — the team stabilized. Delivery followed from that, not the other way around.

How do you handle competing priorities when multiple projects need your attention simultaneously?

My starting point is that competing priorities are almost always a sequencing problem, not a capacity problem. The instinct when everything needs attention simultaneously is to context-switch constantly — but that's the worst possible response. Context switching doesn't distribute your attention, it degrades it. My job is to sequence deliberately so each workstream gets focused attention at the right moment, not fragmented attention all the time.

The clearest example was the paid hints initiative at Ancestry. I was simultaneously managing three workstreams that all needed me in different ways. The first was a two-week validation experiment — lightweight, fast-moving, needed me to make quick scope decisions and keep it from growing beyond its purpose. The second was the full infrastructure design running in parallel — slower, more architectural, needed me in specific review points with the architects but not constantly. The third was cross-team coordination across five engineering teams — relationship-driven, needed consistent cadence but not daily attention.

The way I kept that from becoming chaos was by being explicit about what each workstream needed from me specifically versus what could run without me. The experiment needed me for scope decisions and go/no-go calls — I protected time for that daily. The infrastructure work needed me at defined checkpoints — I set those explicitly with the architects so they weren't waiting on me and I wasn't hovering over them. The cross-team coordination needed a reliable weekly cadence — I set standing syncs and made them non-negotiable.

The second principle I applied was protecting my team from the competing priorities even when I was carrying them myself. The engineers on the infrastructure build didn't need to know the experiment might invalidate their work. I held that uncertainty at my level until the experiment came back positive — at which point I had data to give them, not anxiety. That's a deliberate choice: I absorb ambiguity so my team can focus.

What competing priorities also revealed was who on my team was ready for the next level. When I was stretched across three tracks, I leaned deliberately on my staff engineers — not because I had no choice, but because it was the right leadership move. I gave them ownership of specific tracks and watched how they handled ambiguity, stakeholder communication, and decisions that didn't have clean answers. The experiment track was run almost entirely by one of my senior engineers. That period of competing priorities became one of my richest opportunities to evaluate readiness for promotion — you see things under pressure that you never see in normal sprints.

The result was that all three tracks moved simultaneously without any of them feeling under-managed. The experiment validated in two weeks. The infrastructure was ready to accelerate the moment results came in. The cross-team alignment was already in place before we needed it. Full implementation wrapped in under a month.

What I've learned about competing priorities is that the question isn't how to give everything equal attention — it's how to give each thing the right kind of attention at the right time. Equal attention is a fantasy. Sequenced, deliberate attention is a system

Describe a time a project failed. What was your role and what did you learn?

The failure I think about most is a production incident we had after deploying a new feature to our onboarding flow. All automated tests passed, all smoke tests passed — and about an hour after deployment we had a cascading failure. A legacy flow used by only 5% of our users was triggering a fatal exception that crash-looped our service instances. Because that flow shared process space with our healthy traffic, the crash loop started starving the 95% of resources. Site-wide degradation within an hour.

My immediate response was correct — I triggered a full rollback within five minutes of identifying the instability, ran a tight RCA with two senior engineers, and we were back to 100% health quickly. That part I handled well.

But here's what I actually failed at — and this is the part that stays with me.

The architectural debt that caused this wasn't invisible. I knew that legacy flow was unmaintained. I knew it shared process space with production traffic. What I never did was frame it explicitly as a blast-radius risk and get it on the roadmap before it forced its way there through an incident. I had the information. I didn't act on it. I let the debt sit because there was always something more urgent, and I told myself the risk was low because the flow only affected 5% of users. That reasoning was wrong — low usage doesn't mean low risk when the failure mode is process-level contamination.

The real failure wasn't the incident. It was the three months before the incident where I had enough information to prevent it and chose not to prioritize it.

What changed afterward was structural. I implemented mandatory canary windows before full promotion, lowered our alerting thresholds to catch low-volume high-severity anomalies that average metrics were smoothing over, and initiated a decoupling project to isolate legacy flows into their own workers. That work displaced two features we'd planned that quarter — I made the case to product using the incident data and they agreed immediately.

Those changes caught two other edge-case bugs before they hit a single real user.

But the process change I'm most proud of is the one that happened in my own thinking. I now maintain an explicit risk register for every project — a living document where I force myself to name the things I'm knowingly deferring and what the consequence of deferral looks like. The cascading failure taught me that deferred risk doesn't disappear. It just waits for the worst possible moment to surface. My job is to make that risk visible before it makes itself visible.

Tell me about a time you aligned multiple teams with conflicting priorities

At Ancestry I was tasked with building new packaging infrastructure that required integrating with six open-source stacks owned by five separate engineering teams — none of whom had my project in their roadmap. Leadership wanted it in three months. The teams pushed back immediately when our pull requests started landing in their queues. From their perspective, we were adding work they hadn't planned for and hadn't agreed to.

The temptation in that situation is to escalate — get leadership to mandate their involvement. I deliberately didn't do that. A mandate creates compliance, not collaboration. And I needed these teams to actually care about the quality of the integration, not just reluctantly approve PRs.

So I started with individual conversations. Before any group meeting, before any shared roadmap, I met with each EM one on one. Not to pitch the project — to understand their world. What were their current priorities? Where was their team stretched? What would make a request from me easy to say yes to? Those conversations gave me two things: real information about where the genuine constraints were, and the beginning of a relationship that made everything that followed easier.

What I found was that the resistance wasn't personal — it was structural. Each team was being asked to absorb work that wasn't in their plan with no adjustment to their existing commitments. So I addressed the structural problem directly. I built extensive documentation covering every stack, every integration point, and every effort estimate — not to prove I'd done my homework, but to make their decision-making as easy as possible. I also coordinated with leadership to get one month carved out specifically for design work before any development began. That gave every team planning visibility instead of being surprised by PRs.

Then I brought all five EMs into a group session to map dependencies and build a shared roadmap together. By that point the individual conversations had already established trust — the group session was about alignment, not persuasion. When everyone could see the full dependency picture, the shared ownership dynamic shifted. It stopped being my project that needed their help and became a shared initiative that required coordinated execution.

We launched in two and a half months — only two weeks past the revised timeline, well ahead of the original three-month deadline that had accounted for none of this complexity. More importantly, the five EMs who'd initially pushed back became active collaborators. Three of them contributed to follow-on work because the process had built genuine trust rather than burning it.

What I learned is that conflicting priorities are almost never really about priorities — they're about planning visibility and the cost of yes. Every team that pushed back was pushing back because we'd asked them to absorb unplanned work with no notice and no support. The moment I gave them visibility, reduced the integration burden, and showed them I understood their constraints — the conflict dissolved. Alignment isn't something you achieve by winning an argument. It's something you create by making collaboration easier than resistance.

At Ancestry I was leading an initiative that required integrating five separate engineering teams, each owning a different piece of the flow. None of them had us in their roadmap. Leadership wanted results in three months. On paper, we were blocked before we started.

My first instinct was not to escalate. Escalation would have created resentment — nobody wants their EM told by leadership to drop everything for another team's project. So I did something different. I went to each EM individually, one on one, before any group meeting happened. Not to ask for help — to understand their world. What were they working on? What was their team's capacity? What would make a request from me easy to say yes to versus hard?

Those conversations gave me two things. First, real information — I could see exactly where the genuine constraints were versus where there was quiet capacity. Second, relationships. By the time I brought everyone into a room together, I wasn't a stranger asking for favors. I was someone who understood their problems.

From there I did three things to systematically reduce the block. First, I mapped every dependency across all five teams into flow diagrams and shared them openly — nobody had seen the full picture before, and making the interdependencies visible created shared ownership of the problem rather than bilateral negotiations. Second, I built a shared integration component on my team's side that cut the implementation burden for other teams by roughly 50%. I made it easy to say yes by reducing what yes actually cost them. Third, I ran a two-week lightweight experiment first — before asking anyone to commit to the full build. When that came back positive, I walked into every conversation with data instead of a request. The dynamic shifted completely. I wasn't asking teams to prioritize my project on faith — the experiment had already proven the business case.

The full implementation wrapped in under a month after validation. The shared component cut testing and integration effort for all five teams by about half. And critically — none of those EMs felt steamrolled. Three of them became active collaborators on the follow-up work because the process had built genuine trust rather than burning it.

What I learned: being blocked by another team is almost never really about the other team. It's about whether you've made it easy enough for them to help you. The teams weren't blocking us — they were just full. My job was to reduce the cost of their yes until it became the obvious choice

How do you handle disagreements with a PM or another EM.

My starting point in any disagreement with a PM is to ask myself one question before the conversation: am I pushing back because I have better information, or because I'm uncomfortable with the decision? Those are very different situations and they require very different responses.

The clearest example was the onboarding rebuild at Ancestry. The PM came in with a three-month deadline for what my team's bottom-up estimation showed was a five-month build. On the surface that's a classic Engineering vs. Product tension — product wants speed, engineering wants quality. But I didn't frame it that way and I didn't want the PM to frame it that way either, because that framing turns a planning conversation into a territorial negotiation.

So before I pushed back on the timeline, I did the work. I ran a full t-shirt sizing session with the team, broke every requirement into tickets, explicitly surfaced the hidden work — integration complexity, end-to-end testing, the open-source dependencies we didn't fully own. I came to the PM with a capacity model, not a complaint. The conversation wasn't 'three months is impossible' — it was 'here's what three months actually buys us, and here's what five months buys us, and here's a third option that gets us to market in three months with a phased delivery model.'

That third option — MVP at RootsTech, MLP two months later, full delivery after — was the real move. I didn't just push back, I proposed a path that gave product what they actually needed, which was a story to tell at RootsTech, not necessarily the complete product. The PM agreed immediately because I'd reframed the conversation from a constraint to a strategy.

Mid-project the disagreement went the other way. When the hint regression surfaced, I came to the PM with a feature swap proposal — move one feature out, pull the fix in. The PM's first instinct was to find a way to keep everything in scope. I held my ground — not because I was being territorial about engineering capacity, but because I had specific data showing that shipping with the regression would hurt our primary success metric. I explained the consequence clearly, gave product the trade-off in their language, and let the data make the argument. They came around quickly.

What I've learned about disagreements with PMs is that the relationship survives conflict just fine — what it doesn't survive is the feeling of being surprised or manipulated. Every disagreement I've had that damaged a relationship was one where someone felt blindsided. Every disagreement that strengthened a relationship was one where both sides felt heard and the decision was made transparently with shared information.

My default is always to bring more data than the conversation needs and fewer opinions than I actually have

Disagreement with Architect

When we were planning the new onboarding application at Ancestry, I ran into significant pushback from our architect during a formal architecture review. His position was clear — build the new application on one of our existing stacks rather than creating something new. On the surface it was a reasonable position. Reusing existing infrastructure reduces risk, leverages known patterns, and avoids the overhead of building from scratch.

But I had a fundamental disagreement with that approach — and I had data to back it.

My concern wasn't philosophical. It was practical and specific. There were two structural problems with building on the existing stack that I didn't think could be engineered around.

The first was deployment dependency. We didn't own that stack — another team did. Every change we needed to push, no matter how small, would require us to open a PR into their codebase and wait for their team to review and deploy it. For a core application serving millions of users on a slow release cadence, that's manageable. For a new onboarding experience that needed to run frequent experiments and iterate quickly based on user behavior, it was a fundamental mismatch. Our deployment frequency requirements were completely different from theirs. We would have been permanently constrained by another team's bandwidth and release schedule.

The second was blast radius and coupling risk. The existing stack served our core application — a product used by the vast majority of our user base. The new onboarding application was fundamentally different. It was designed for a much smaller, newer audience, with different traffic patterns, different experiment cadences, and different risk tolerance. Putting both on the same stack meant that any production issue in our new application — and new applications always have early production issues — could potentially impact the core product. I didn't want to introduce that coupling. The whole point of building something new was to be able to move fast and learn fast without putting the core experience at risk.

I didn't walk into the architecture review with opinions. I walked in with a formal proposal covering four specific dimensions. First, expected traffic analysis for the new application versus the core product — showing how different the load profiles were. Second, the experiment pipeline we had planned for the new experience — showing how frequently we anticipated needing to push changes and why that cadence was incompatible with the existing team's release rhythm. Third, a blast radius analysis — showing what the impact of a production error in the new application would be if it was isolated versus if it was coupled to the core stack. Fourth, a tight coupling risk assessment — showing the specific scenarios where changes to the core stack could break the new application and vice versa.

The architect pushed back hard on several of my assumptions. He felt the deployment dependency concern was overstated — that we could negotiate a faster release cadence with the owning team. I acknowledged that was possible but argued it created an ongoing coordination overhead and a permanent external dependency on another team's priorities. He also felt the blast radius concern was manageable with proper feature flagging. I acknowledged that too — but argued that feature

flagging adds complexity and that architectural isolation was a cleaner solution than mitigation on top of a coupled system.

We didn't reach full agreement in that meeting. The architect was about 80% convinced — he accepted the deployment dependency argument clearly, was partially convinced on the blast radius concern, but remained skeptical that the overhead of a new stack was justified. What I walked away with was approval to proceed — with the explicit understanding that we would run early experiments quickly and use the results to validate the architectural decision in practice.

The launch was successful and on time. The other team continued their work on the core stack without any interference from our deployments. The experiment cadence we ran in the first weeks after launch made it increasingly evident that the architectural separation had been the right call — we were pushing changes at a frequency that would have been impossible under the shared stack model.

The architect never became a full enthusiastic advocate for the decision. But he told me afterward that he was glad we'd had the discussion — that the formal proposal and the data had given him confidence the decision had been made rigorously rather than casually. The relationship stayed intact. And in some ways his skepticism made the outcome stronger — it forced me to build a more rigorous case than I would have otherwise.

What I learned from that experience is that when you're pushing back against someone with institutional knowledge and authority, the worst thing you can do is make it a battle of opinions. Opinions create defensiveness. Data creates the conditions for genuine reconsideration. The architect wasn't being unreasonable — he was applying a valid heuristic from years of experience. My job was to show him specifically why this situation was different from the pattern his heuristic was built on. That required precision, not persuasion.

How do you build relationships with stakeholders you don't directly work with?

My approach to building relationships with stakeholders I don't directly work with starts from one principle: I never ask for something before I've created something. The moment you show up to a new relationship with a request, you've defined yourself as a cost. The goal is to show up as a value first.

The clearest example is the user journey analytics tool I built at Ancestry. This wasn't an assigned project — I saw a problem nobody was solving and decided to own it. The problem was that stakeholders across the company kept asking 'what happened to this user journey?' and nobody could answer confidently. Our logging captured errors but gave no coherent picture of what a user actually did during a session.

To build this tool I needed relationships with three groups of people who had no reason to prioritize me: the internal New Relic team, the product team, and my VP.

With the New Relic team I led with curiosity, not a proposal. I reached out to explore what was possible with our existing data — genuinely asking what they'd already tried, what constraints existed, what they'd want to see built. That conversation shaped the entire technical approach. When I came back with a proof of concept, they were already invested in it because they'd helped define it. I also made a deliberate call to get their explicit sign-off that no user data was at risk before going further — not because I was required to, but because protecting their trust was more important than moving fast.

With the product team I approached it differently. I didn't ask them to review a tool — I asked them what questions they wished they could answer about user behavior. That framing changed the conversation completely. Instead of evaluating my work they were co-designing it. The dwell-time data that became one of the tool's most valuable features came directly from that conversation — it wasn't something I'd planned, it was something they told me they needed.

With my VP I used progressive transparency. I didn't wait for a finished product to get visibility. I demoed early, demoed often, and framed every demo around the business problem being solved, not the technical implementation. By the time I presented at the all-hands meeting, my VP had seen the work three times and had already formed a positive view of it. The all-hands wasn't a pitch — it was a confirmation.

The tool ended up being adopted by four teams as a pilot and eventually spread across the broader engineering org as an internal open-source platform. Bug investigation time dropped by 75%. None of that adoption happened because I asked people to use it — it happened because I'd built relationships with the right people before I needed anything from them.

What I've learned about stakeholder relationships is that the ones that matter most are the ones you build before you need them. By the time you're in a situation where you need someone's support, it's too late to start the relationship. The New Relic team, the product team, my VP — I

invested in those relationships during the build, which meant I had genuine advocates at launch rather than neutral observers.

Describe a time you had to influence a decision you had no authority over.

Preference on Paid hints but if that story is already used then use Journey logger

How do you handle situations where another team's work negatively impacts yours?
About six weeks into our onboarding rebuild at Ancestry, we hit a regression we hadn't caused. Hint generation rates weren't matching the legacy flow — and hints are the core hook of the Ancestry experience, one of our primary success metrics for the launch. When I dug into the root cause with a senior engineer, we found it: a fundamental change in how user data was being ingested upstream had altered how hints were being generated. That change was owned by another team. They hadn't done anything wrong — it was a legitimate architectural decision on their end. But the downstream impact on our flow was significant.

This is the moment where a lot of EMs make a mistake. The temptation is to escalate immediately — flag the other team as the source of the problem and make it their responsibility to fix it. I deliberately didn't do that. Here's why: escalation in that moment would have framed the situation as a conflict. The other team would have become defensive. And I needed their help, not their defensiveness.

So I did three things in sequence.

First, I made sure I fully understood the impact before I picked up the phone. I paired directly with a senior engineer to quantify exactly what the regression meant in terms of our launch metrics — not in technical terms but in business terms. Hint generation rate down by X, projected impact on build-through rate, projected impact on RootsTech launch. I wanted to walk into any conversation with data, not alarm.

Second, I approached the other team's EM as a partner, not an adversary. I framed the conversation around the shared problem — we have a launch in six weeks, here's what we're seeing, here's the data, I'd like to understand what changed on your end so we can figure out the best path together. That framing completely changed the dynamic. They weren't being accused of breaking something — they were being invited to help solve something.

Third, I separated the relationship conversation from the solution conversation. I didn't ask them to fix it. I took ownership of the solution on my side — proposed a feature swap to product that brought the hint fix into MVP scope without asking the upstream team to roll back their architectural decision. Their change was valid. My job was to absorb the impact on my end, not make it their problem to undo.

The MVP launched on time at RootsTech. Hint generation rate came in 2% above the legacy flow — we reversed the regression entirely. The upstream team's EM actually reached out after launch to say it was one of the cleanest cross-team dependency resolutions they'd experienced.

What I learned is that when another team's work impacts yours negatively, the most important decision you make in the first five minutes is how you frame the conversation. If you frame it as 'you broke something of mine' you get a defensive negotiation. If you frame it as 'we have a shared problem and I want to solve it together' you get a collaborator. The technical fix was straightforward. The relationship management was the real work.

What would you have done if the upstream team's change had been a mistake — something they should have caught before shipping?
I would have had the same opening conversation — understand what happened, share the impact data, frame it as a shared problem. The difference is that in the solution conversation I would have been clearer that the fix needed to happen on their side, not mine. But I'd still lead with data and partnership rather than blame — because even when someone else is clearly at fault, the way you hold them accountable determines whether you have a functional working relationship afterward or a damaged one.

Why was it not affecting the legacy flow

The legacy flow and the new flow were pulling user data differently. The legacy onboarding had been built when user data was structured in a specific way — it knew exactly where to find the information it needed to generate hints, and that hadn't changed for years. Our new flow was built on a more modern data ingestion approach that the upstream team had architected — but the way that new approach assembled user data meant certain fields that hint generation depended on were either arriving later in the session, arriving in a different format, or in some cases not being populated at all at the point in the flow where hints were triggered.

The legacy flow never touched that new ingestion layer — it was completely decoupled from it. So the upstream change had zero impact on legacy. Our new flow was the first system to actually consume that new ingestion approach at the hint generation stage, which is why we were the first to surface the problem.

The irony is that we discovered a gap in the upstream team's new architecture that nobody else had caught yet — because nobody else was depending on it at that specific point in the user journey. So while the regression hurt us short term, surfacing it actually helped the broader org. The upstream team tightened their data population guarantees as a result, which prevented the same issue from hitting other teams later.

Tell me about a time you had to represent your team's interests while compromising for the greater org good

At Ancestry, my team was handed a project to build new packaging infrastructure across six unfamiliar open-source stacks, owned by five separate teams, in three months. When I sat down with my team to do honest bottom-up estimation, the number that came back was six months. Not because the team was being conservative — because the complexity was real. Six stacks we'd never worked with, no finalized designs, cross-team dependencies that nobody had mapped yet.

My team was already stretched. They were maintaining our existing onboarding system while being asked to build its replacement. The three-month deadline wasn't just aggressive — it was the kind of deadline that burns people out and drives attrition if you don't handle it right.

So I had two responsibilities pulling in opposite directions. The first was to my team — to represent honestly what this work would cost them and make sure leadership understood the real capacity picture before committing. The second was to the org — leadership had a legitimate business case, contractors had validated the revenue potential, and five other teams were waiting for direction. The org needed this to move.

I did three things to hold both responsibilities at once.

First, I represented my team's reality with data, not emotion. I built a granular effort and risk document covering every stack, every integration dependency, every piece of hidden work — end to end testing, open source contribution requirements, the absence of finalized designs. I presented a six month estimate to leadership not as a negotiating position but as a capacity model. My team deserved to have their reality communicated accurately, not softened to make leadership comfortable.

Second, I made a compromise that cost my team short term but protected them long term. I advocated for one dedicated month of design work before any development began. From my team's perspective, that felt like a delay — they were ready to build and wanted to move. But I knew that starting development across six unfamiliar stacks without clear designs would create rework, cross-team confusion, and the kind of mid-flight chaos that burns people out faster than a tight deadline ever could. I explained this to my team directly: the design month isn't a luxury, it's protection. We're absorbing a short term delay to avoid a much worse situation in month two.

Third, I was completely transparent with my team about what we were signing up for and why. I didn't pretend the timeline was comfortable or that leadership had given us everything we'd asked for. I told them directly: here's what we negotiated, here's what we couldn't change, here's why I think we can do this, and here's what I'm going to do to protect you through it. I wanted them bought in, not blindsided.

The project launched in two and a half months — only two weeks past our revised timeline, well ahead of the original three month deadline that had accounted for none of the complexity. The design month paid off exactly as I'd predicted — cross-team conflicts during development were

significantly lower than they would have been without it. And critically, not one engineer left during the project despite the pressure.

What I learned is that representing your team's interests doesn't mean winning every negotiation on their behalf. Sometimes it means making sure the trade they're being asked to make is an honest one — that they understand what they're signing up for, why it matters, and that you've done everything in your power to protect them within the constraints you couldn't change. A team that feels represented can handle a hard situation. A team that feels sacrificed cannot.

if this story is already used, tech debt Friday story can also be used here since we compromised product velocity with tech-debt and learning new things

How do you create alignment between Engineering and Product when roadmaps conflict?

My view is that roadmap conflicts between Engineering and Product are almost always a symptom of the same root cause — Engineering and Product are operating on different time horizons without a shared visibility mechanism. Product is thinking about what users need. Engineering is thinking about what's technically feasible and how long it takes. When those two conversations happen sequentially instead of simultaneously, conflict is inevitable.

The clearest moment that crystallized this for me was when our entire product and UX org was laid off and a new product team came in with a different vision. My engineering team had bandwidth but no direction. The new product team had direction but needed time to finalize it. In the traditional model we would have waited — requirements come first, engineering follows. That waiting would have cost us a month of productive build time and created a cliff edge where engineering had to suddenly absorb fully formed requirements with no runway.

I decided to break that model deliberately.

I set up individual one on ones with the incoming PMs as soon as they were assigned — not to get requirements, but to understand their directional thinking before anything was finalized. Those conversations gave me enough signal to identify that tree building functionality was central to their vision. I made the call to start building toward that target without final specs, knowing some work might be throwaway. Before moving forward I brought my manager in, explained the approach and the tradeoff, and got his support. I was equally transparent with my team — told them directly that some of this work might get thrown away. I wanted them bought in, not blindsided. I also communicated clearly to the product team that our prototype work would not constrain or influence their independent vision building process. That boundary was critical — the moment engineering's early work starts feeling like a constraint on product thinking, the relationship breaks down.

From there I synced continuously with product as their thinking evolved and pivoted the implementation with each new signal. I engaged architects throughout to validate feasibility. When final requirements and designs landed a month later, my team already had a working solution close to target. Build time dropped from an estimated four months to three. Conversion improved 10%, NPS improved 15%.

But the most important outcome wasn't the delivery metric — it was the structural change I made afterward. I introduced early vision meetings with product as an ongoing practice. Not just for this project, but as a standing mechanism. Product shares directional thinking before requirements are complete. Engineering flags feasibility concerns and capacity constraints before commitments are made. If bandwidth allows, we begin prototyping against directional signals rather than waiting for final specs.

That practice changed the entire dynamic between Engineering and Product on my team. Roadmap conflicts didn't disappear — but they moved earlier in the process, where they're cheap to resolve, rather than surfacing mid-sprint where they're expensive and damaging.

What I learned is that Engineering and Product alignment isn't something you achieve in a negotiation. It's something you build through a shared visibility rhythm. The earlier both sides can see each other's constraints and thinking, the fewer surprises there are when commitments get made. Waiting for complete requirements is often the wrong default. Enough directional signal is enough to start building meaningfully — and starting early is what creates the relationship that makes alignment possible.

What do you do when Product isn't willing to share directional thinking early — when they want to come to Engineering with fully formed requirements

I've encountered that reluctance and it usually comes from one of two places — either product has been burned before by engineering pushing back on half-formed ideas, or they feel like sharing early thinking gives engineering too much influence over their vision. I address both directly. For the first, I establish an explicit norm upfront: early conversations are exploratory, not commitments. Engineering won't hold product to anything said before requirements are final. For the second, I do exactly what I did with the new product team — I make it explicit that prototype work won't constrain their vision. That boundary has to be real and consistently honored, not just stated once. When product trusts that early sharing doesn't mean early commitment, the reluctance usually dissolves.

Describe a time you helped resolve an org-level conflict that wasn't yours to own.

User Journey Logger
At Ancestry, a frustrating question came up constantly across multiple teams — 'what happened to this user journey?' — and nobody could answer it confidently. Our logging in New Relic only captured errors. It gave no coherent picture of what a user actually did during a session. Stakeholders were filing spike tickets just to investigate basic behavior questions. Bug diagnosis was taking four times longer than it should have. The closest third party solution was FullStory at a million dollars.

This wasn't my problem to solve. I was an engineer on a product team with my own roadmap and my own delivery commitments. Nobody asked me to fix it. But I kept watching teams across the org waste time on problems that a better diagnostic tool would have solved in minutes — and at some point I decided that waiting for someone else to own it was its own kind of choice.

So I chose to own it instead.

My first move was to understand the constraints before building anything. I reached out to the internal New Relic team to explore what was possible with our existing data infrastructure. They suggested grouping requests by request ID — I evaluated that approach and determined it wasn't sufficient to reconstruct a full user journey. So I built a proof of concept myself, reading logs from New Relic and generating custom dashboards showing what events a user triggered, what pages they viewed, how they moved between pages, and how long they spent on each one.

Before going further I brought the POC back to the New Relic team for two specific reasons — to validate it against company policy and get explicit sign-off that no user data was at risk. That step was non-negotiable for me. Moving fast on something that touched user data without the right stakeholders signed off would have created a much bigger org-level problem than the one I was solving.

I also met with the product team to understand what insights would actually be useful to them — specifically whether dwell-time data could support experiment design. That conversation shaped the tool significantly. I made a deliberate call to scope out video session replay entirely, keeping the project focused, shippable quickly, and free of security risk.

I built custom funnels and cohort analysis into the platform, then integrated it with Statsig, Ancestry's experimentation platform, so user journey data and A/B test results could live in a single dashboard. I demoed progress continuously to my manager and VP to maintain visibility and surface concerns early. When the tool was ready I demoed it at an all-hands meeting, onboarded four teams as a pilot, then presented at Ancestry's monthly engineering demo — which drove adoption across the broader engineering org.

I made the project open-source internally and handed it off to the New Relic team so they could build on top of it and maintain it beyond my involvement.

The results were significant. Spike tickets created to investigate bugs dropped by 75%. Time to diagnose bugs fell to a quarter of what it previously took. The product team identified several new experiment ideas directly from the dwell-time data. The tool spread well beyond my team and became an internal open-source platform with contributions from multiple teams. Ancestry got a practical alternative to a million dollar vendor product.

What I learned from this experience is that org-level problems rarely get solved by the person whose job it is to solve them — because often nobody's job it is. They get solved by whoever decides the cost of the problem is higher than the cost of their own discomfort in stepping outside their lane. The moment I made that decision, everything else followed. The technical work was hard but straightforward. The harder thing was giving myself permission to own something that wasn't mine to own.

How do you keep communication healthy across distributed or cross-functional teams heartbeat calls

My view on communication across distributed or cross-functional teams is that most communication problems aren't caused by people communicating badly — they're caused by systems that make good communication harder than it needs to be. My job as an EM is to build the structures that make healthy communication the path of least resistance, not a personal virtue people have to remember to practice.

The most complex communication challenge I've managed was the paid hints initiative at Ancestry — five engineering teams, each owning a different piece of the flow, none of whom had shared visibility into each other's dependencies, all working under a tight deadline with competing roadmap priorities. The default in that situation is bilateral communication — each team talking to adjacent teams when something comes up, information moving slowly and inconsistently, blockers sitting invisible until they become crises.

I replaced that default with three structural mechanisms.

The first was dependency visibility before anyone started building. I mapped every integration point across all five teams into flow diagrams and shared them openly — not as a project management artifact but as a communication tool. Nobody had seen the full picture before. The moment everyone could see how their work connected to everyone else's, the communication dynamic shifted. Teams stopped waiting to be told about downstream impacts and started proactively flagging upstream risks. Shared visibility created shared ownership of the communication problem.

The second was a weekly cross-team sync designed specifically around one question: what's in the way? Not what did you complete, not what are you working on — what's blocking you right now and what's likely to block you next week? That distinction is everything. Status reporting is backward looking — it tells you what happened. Blocker surfacing is forward looking — it tells you what's about to happen. I structured the sync explicitly around the second question and kept it to thirty minutes. The constraint forced people to surface only what actually mattered.

The third was individual relationships before group communication. Before the first cross-team sync, I had met individually with every EM. Those one on ones weren't about the project — they were about understanding each person's world, their constraints, their communication preferences. When you know someone as a person before you know them as a stakeholder, the group communication that follows is completely different. People flag things earlier, frame things more generously, and resolve things faster when they're talking to someone they have a relationship with rather than a name on a calendar invite.

Those three mechanisms — shared dependency visibility, blocker-focused cadence, individual relationships first — kept communication healthy across five teams for the full duration of the project. Full implementation wrapped in under a month after validation with no major communication breakdowns.

But the mechanism I'm most proud of building came from a different experience — the communication breakdown between Engineering and Product that happens when requirements are treated as the starting point of the relationship rather than the midpoint. After navigating a period where our entire product org was replaced and my team had no direction for two months, I introduced early vision meetings as a standing practice. Product shares directional thinking before requirements are complete. Engineering flags feasibility and capacity constraints before commitments are made. Both sides can see each other's constraints early enough that conflicts surface when they're cheap to resolve rather than when they're expensive and damaging.

That practice changed the communication rhythm between Engineering and Product permanently — not just for one project but as an ongoing operating model.

What I've learned about communication health across teams is that it degrades by default. Without deliberate structure it reverts to reactive, bilateral, status-focused communication that surfaces problems too late. My job isn't to be a good communicator personally — it's to build systems where good communication is what happens naturally, even when I'm not in the room.

Tell me about a time a peer EM or stakeholder pushed back hard on your plan. How did you respond?

When we were planning the new onboarding application at Ancestry, I ran into significant pushback from our architect during a formal architecture review. His position was clear — build the new application on one of our existing stacks rather than creating something new. On the surface it was a reasonable position. Reusing existing infrastructure reduces risk, leverages known patterns, and avoids the overhead of building from scratch.

But I had a fundamental disagreement with that approach — and I had data to back it.

My concern wasn't philosophical. It was practical and specific. There were two structural problems with building on the existing stack that I didn't think could be engineered around.

The first was deployment dependency. We didn't own that stack — another team did. Every change we needed to push, no matter how small, would require us to open a PR into their codebase and wait for their team to review and deploy it. For a core application serving millions of users on a slow release cadence, that's manageable. For a new onboarding experience that needed to run frequent experiments and iterate quickly based on user behavior, it was a fundamental mismatch. Our deployment frequency requirements were completely different from theirs. We would have been permanently constrained by another team's bandwidth and release schedule.

The second was blast radius and coupling risk. The existing stack served our core application — a product used by the vast majority of our user base. The new onboarding application was fundamentally different. It was designed for a much smaller, newer audience, with different traffic patterns, different experiment cadences, and different risk tolerance. Putting both on the same stack meant that any production issue in our new application — and new applications always have early production issues — could potentially impact the core product. I didn't want to introduce that coupling. The whole point of building something new was to be able to move fast and learn fast without putting the core experience at risk.

I didn't walk into the architecture review with opinions. I walked in with a formal proposal covering four specific dimensions. First, expected traffic analysis for the new application versus the core product — showing how different the load profiles were. Second, the experiment pipeline we had planned for the new experience — showing how frequently we anticipated needing to push changes and why that cadence was incompatible with the existing team's release rhythm. Third, a blast radius analysis — showing what the impact of a production error in the new application would be if it was isolated versus if it was coupled to the core stack. Fourth, a tight coupling risk assessment — showing the specific scenarios where changes to the core stack could break the new application and vice versa.

The architect pushed back hard on several of my assumptions. He felt the deployment dependency concern was overstated — that we could negotiate a faster release cadence with the owning team. I acknowledged that was possible but argued it created an ongoing coordination overhead and a permanent external dependency on another team's priorities. He also felt the blast radius concern

was manageable with proper feature flagging. I acknowledged that too — but argued that feature flagging adds complexity and that architectural isolation was a cleaner solution than mitigation on top of a coupled system.

We didn't reach full agreement in that meeting. The architect was about 80% convinced — he accepted the deployment dependency argument clearly, was partially convinced on the blast radius concern, but remained skeptical that the overhead of a new stack was justified. What I walked away with was approval to proceed — with the explicit understanding that we would run early experiments quickly and use the results to validate the architectural decision in practice.

The launch was successful and on time. The other team continued their work on the core stack without any interference from our deployments. The experiment cadence we ran in the first weeks after launch made it increasingly evident that the architectural separation had been the right call — we were pushing changes at a frequency that would have been impossible under the shared stack model.

The architect never became a full enthusiastic advocate for the decision. But he told me afterward that he was glad we'd had the discussion — that the formal proposal and the data had given him confidence the decision had been made rigorously rather than casually. The relationship stayed intact. And in some ways his skepticism made the outcome stronger — it forced me to build a more rigorous case than I would have otherwise.

What I learned from that experience is that when you're pushing back against someone with institutional knowledge and authority, the worst thing you can do is make it a battle of opinions. Opinions create defensiveness. Data creates the conditions for genuine reconsideration. The architect wasn't being unreasonable — he was applying a valid heuristic from years of experience. My job was to show him specifically why this situation was different from the pattern his heuristic was built on. That required precision, not persuasion

How do you identify and close skill gaps on your team?

My approach to skill gaps runs on two tracks simultaneously — identifying individual gaps through ongoing observation and creating structural opportunities for the whole team to grow in directions the business will need.

On the individual side, I don't wait for performance reviews to surface gaps. The signals are almost always visible earlier if you're looking in the right places. In my 1:1s I watch for three specific things: whether an engineer is asking questions that show they understand the why behind their work, whether they're engaging in design conversations or going quiet, and whether they're taking ownership of outcomes or just completing tasks. Those three behaviors tell me more about where someone is than any performance metric.

The clearest example of proactive gap identification was during the period after our entire product and UX org was laid off at Ancestry. My team of twelve suddenly had no product direction. That situation — as destabilizing as it was — gave me an unusually clear view of where each engineer's capabilities actually were. When external scaffolding disappears and engineers have to decide for themselves what to work on and how to stay productive, you see exactly who has the initiative, the domain understanding, and the learning instinct to operate in ambiguity.

What I saw was a skill gap that was going to matter enormously within twelve months — AI and LLM capabilities. Not because anyone was failing, but because the direction technology was heading was obvious and my team had zero exposure to it. I didn't wait for a roadmap item to force the issue. I made a deliberate call to use the ambiguous period strategically. I directed the team toward AI capability building — starting by building things myself, using LangChain to build a chatbot so I could understand how these tools actually worked before asking my team to engage with them. Then I created a structure for the team to build AI projects during our protected Friday learning time, with demos so everyone could learn from each other's exploration.

Several engineers developed capabilities that became directly applicable to our work within months. We evolved from individual learning into a collaborative project — what became Ancestry University — which the product team eventually adopted as a feature concept.

The structural mechanism I rely on most for closing gaps at the team level is protected learning time with accountability. Not just 'go learn something' — but dedicated time with a demo structure so learning is visible and social. Engineers learn faster when they're building something real and showing it to peers than when they're watching tutorials alone. The demo creates the deadline. The deadline creates the output.

What I've learned about skill gaps is that the ones that hurt you most are never the ones you know about — they're the ones you haven't named yet. My job is to be twelve to eighteen months ahead of the business in identifying what capabilities my team will need, and to create the conditions for those capabilities to develop before they're urgently required.

How do you identify gaps in engineers who are delivering well but may be plateauing?

High performers are actually harder to read because their delivery masks the ceiling. The signal I watch for is whether they're growing their impact radius — are they influencing decisions beyond their own tickets, mentoring others, shaping the architecture? A senior engineer who's executing perfectly but staying in their lane is a retention risk as much as a development gap. I have explicit conversations about scope expansion with high performers — not 'you need to do more' but 'here's what the next level looks like and here's one concrete thing you could own that would get you there.

What if an engineer doesn't want to develop in the direction the business needs?

I separate the conversation into two parts. First I try to understand what they do want — sometimes resistance to a direction is really just preference for a different one, and if that direction is also valuable I'll find a way to create space for it. But if there's a genuine mismatch between where the engineer wants to grow and where the business needs them to develop, I'm honest about it: this is what the role requires going forward, here's the support I can provide, and here's what it means for your trajectory if we can't close the gap. I'd rather have that conversation eighteen months early than have it in a performance review.

Walk me through how you've handled a low performer end to end

The Story: Turning a Struggling Peer into a Staff Engineer

The most complete low performer story I have spans two years and ends with a promotion to Staff Engineer — which tells you something about how I think about performance management.

When I transitioned into an EM role at Ancestry, I inherited a direct report who had joined the company at the same time I had. We'd been peers. In our very first 1:1 as manager and direct report, he told me he expected a promotion soon. In reality, he was trending toward a performance improvement plan. There was a massive gap between his self-perception and his actual impact — and I was the first person in a position to close that gap honestly.

That first conversation was the hardest thing I did in my first month as an EM. I didn't ease into it. I told him directly — you're not on a promotion track, and here are the specific examples that show me why. I kept everyone else's feedback anonymous to protect psychological safety, but I was clear about the business impact of what I was observing. The goal of that conversation wasn't to demoralize him — it was to move past the denial phase and into problem-solving. You can't fix a gap someone doesn't know exists.

What happened next is what I'm most proud of. Rather than defaulting to a PIP, I treated it as a diagnostic problem first. Through our conversations I realized the issue wasn't his coding ability — it was a lack of domain and architectural context. He didn't understand how our complex data systems were interconnected. Because of that gap, he couldn't contribute meaningfully to high-level design conversations, couldn't challenge product requirements, and couldn't see the downstream impact of his technical decisions. He wasn't underperforming because he was disengaged — he was underperforming because he was lost in a system he didn't understand.

So I built a structured 90-day knowledge sprint specifically for him. Beyond our regular 1:1s, I dedicated fifteen minutes each week to a technical deep dive — I'd explain a piece of the architecture and then give him a scavenger hunt task. Not 'read this documentation' — but 'find the specific data flow between these two systems and tell me where the bottleneck is.' The goal was to force him to get his hands dirty with the architecture rather than just hear about it. I didn't give him answers. I pointed him to the systems where he could find them himself — because the skill I was building wasn't knowledge, it was the instinct to go find answers independently.

The milestones were clear. At thirty days he started asking the right questions in grooming sessions — showing he finally understood the why behind the tickets. At ninety days he exceeded my expectations — he successfully led a cross-team architectural discussion that six months earlier he would have avoided entirely.

Over the next two years he continued on that trajectory. I promoted him to Staff Engineer. Today he's one of the strongest mentors on the team — specifically helping new hires navigate the same

domain complexity he once struggled with. The person who was closest to a PIP in my first month became a cultural asset to the engineering org.

What I learned from this experience shaped how I think about low performance entirely. The first question I ask now is never 'is this person performing' — it's 'do I understand why they're not performing.' Most underperformance has a root cause that's fixable if you find it early enough. The PIP is a last resort, not a first response. The radical candor conversation is the first response — because without that, nothing else works.

What would you have done if he hadn't responded to the development plan?

I had a clear internal threshold — if at ninety days he wasn't asking better questions in design conversations, I would have had a second direct conversation: here's what I've observed, here's what I've invested, here's what I need to see in the next thirty days, and here's what happens if we don't get there. The PIP would have been the next step — not as a punitive measure but as a formal documentation of exactly what the bar is and what support is available. What I wouldn't have done is let it drift indefinitely. Ambiguity is cruel to a struggling engineer — clarity is the kindest thing you can give them even when the message is hard.

How did you manage the rest of the team while this was happening?

I kept everything confidential — the team never knew the specifics of what was happening in our 1:1s. What they did see was his behavior changing. When he started asking better questions in grooming, when he led that cross-team architectural discussion at ninety days — those were visible to everyone. The team's perception shifted organically as his performance shifted. I didn't need to manage the narrative because the results spoke for themselves. The one thing I was careful about was making sure the investment I was putting into his development wasn't visibly disproportionate to what I was offering others — I made sure the Tech Deep Dive format was available as a mechanism for anyone who wanted to use it.

How do you calibrate and retain your top engineers?

Calibration and retention are connected problems for me — an engineer who is accurately calibrated and sees a clear path forward almost always stays. The ones who leave are almost always either undervalued and don't know it, or overvalued and about to hit a wall they didn't see coming.

On calibration, my philosophy is that the review cycle should never be the first time an engineer hears where they stand. I maintain a running mental model of where each engineer is relative to the bar for their level — and I update it continuously through what I observe, not what I'm told. The most important calibration work happens in the months before the review, not during it.

The clearest example was a senior engineer on my team whose technical expertise was genuinely exceptional — deep React knowledge, real architectural instinct. But his contribution pattern was creating friction rather than value. He was leaving extensive PR comments on style issues, blocking merges, and the team was starting to avoid opening PRs at all. Sprint velocity dropped 30% over two sprints.

The calibration problem here was subtle. On paper he was performing well. In practice his impact was negative — not because of his skills but because of how he was deploying them. I had to recalibrate what success looked like for him specifically. His expertise was real. The outlet for it was wrong.

I had a direct conversation. I told him the impact his review style was having on team velocity and morale — specific numbers, specific examples. He pushed back initially. I stayed patient and kept explaining the impact until he became genuinely open to something different. Then we co-created a solution together — a PR tagging system that gave developers clarity on what actually blocked a merge, and a shift in his role toward active mentorship rather than gate-keeping. I also introduced Husky to automate the style checks he was doing manually, which removed the friction entirely at the source.

The result was that he found a better outlet for his expertise — one that actually reflected his real value to the team. PR merge time dropped to within 24 hours. Velocity increased 30%. And in the next review cycle I could make a calibration argument for him based on his mentorship impact, not just his individual technical output. He went from being a friction point to being a force multiplier.

On retention — calibration is necessary but not sufficient. Engineers who know where they stand also need to feel that the work itself is worth staying for. The most dangerous retention risk isn't an underperforming engineer — it's a high performer who is bored, unchallenged, or feels like they're grinding without growing.

The mechanism I've built that has the highest retention impact is protected learning time with a demo structure. On one team I introduced no-meeting Fridays specifically to create space for engineers to learn new technologies, explore ideas beyond the current roadmap, and build things that excited them. I modeled it myself — I used Fridays to learn LangChain and build a chatbot, and

demoed it to the team. Engineers followed. Several built AI projects that became directly applicable to our work within months. The team eventually built something we called Ancestry University — a collaborative project that the product team adopted as a feature concept.

The retention signal I watch most closely is creative energy. A team that's generating new ideas, proposing experiments, suggesting improvements to their own tools — that's a retained team. A team that's executing tasks without curiosity or initiative — that's a team where the best people are already mentally interviewing elsewhere.

What I've learned about retaining top engineers is that compensation matters at the floor but not at the ceiling. Once someone is fairly paid, the reasons they stay are growth, autonomy, and the feeling that their work matters. My job is to make sure all three are present — and to notice early when any one of them is eroding."

What does your hiring process look like and how do you assess culture add vs culture fit?

"My hiring philosophy starts from a deliberate rejection of the term culture fit. Fit is a recipe for building a team that thinks the same way, has the same blind spots, and makes the same mistakes collectively. What I hire for instead is values alignment plus capability diversity — and those are two very different filters.

Values alignment is non-negotiable. There are specific things every engineer on my team has to genuinely care about — shipping quality work, being honest about what they don't know, treating colleagues as adults, and defaulting to collaboration over territory. I don't compromise on those. But within that boundary I actively look for people who will challenge how we do things, bring perspectives the team doesn't have, and make us collectively smarter by being different from us.

The question I rely on most in every interview loop to assess this is one I developed from watching engineers navigate a specific real-world tension: 'A PM comes to you with a requirement that's logically correct from a product perspective but isn't achievable with our current infrastructure. What do you say?'

The wrong answers reveal themselves immediately. Some candidates say they'd accept the work and figure it out — that's a people-pleaser who will overpromise and create debt. Some candidates say they'd push back and explain why it can't be done — that's an engineer who optimizes for their own comfort over business outcomes. Neither is the person I want.

The answer I'm looking for is: this doesn't exist today, but let's work on it together — we'll build it in pieces and test the waters first. That answer tells me three things simultaneously. First, technical honesty — they're not pretending a gap doesn't exist. Second, product partnership instinct — they see the PM as a collaborator, not a requester. Third, execution pragmatism — they default to incremental validation rather than big bang delivery.

The follow-up I always ask after their answer is equally revealing: 'How do you decide what to test first?' A strong candidate talks about validating the riskiest assumption earliest. A weaker candidate talks about building the easiest piece first. Those are completely different risk philosophies — and the difference tells me more about operating model fit than any technical assessment.

Beyond that question, my process has three other components. First, I always include a structured debrief where every interviewer maps their signal to specific behaviors, not impressions. 'I liked them' is not debrief feedback. 'They challenged my assumption about the architecture and backed it with data' is debrief feedback. Second, I specifically watch for candidates who disagree with something during the interview — not combatively, but with evidence and intellectual confidence. The best hire I can make is someone who pushes back on me thoughtfully in the interview itself. Third, I weight the debrief question 'what would this person add that we don't currently have' more heavily than 'would this person fit in.' The first question builds the team. The second question replicates it.

What I've learned about hiring is that the candidates who feel most comfortable in an interview are often the ones most practiced at performing comfort — not the ones who will actually thrive in ambiguity. The question I developed cuts through that performance because there's no safe answer. You either have the instinct or you don't."

Follow-up: How do you handle a situation where the team loves a candidate but you have a concern?

"I take the concern seriously regardless of team enthusiasm — because enthusiasm is contagious and can mask real signal gaps. I go back to the specific behavior that triggered my concern and ask whether any interviewer saw counter-evidence. If nobody saw counter-evidence, the concern stands. I'd rather have a hard conversation in the debrief than spend six months managing a mis-hire. I also distinguish between a concern about skills — which can be developed — and a concern about values or operating model — which almost never changes after hiring. Skills concerns I can live with if growth instinct is present. Values concerns I cannot."

Tell me about a time you had to reshape or rebuild a team.

The most complete rebuild I've led happened when our entire product and UX org was laid off overnight during a strategic pivot at Ancestry. My engineering team of twelve went from daily collaboration with product managers and designers to complete silence — no requirements, no roadmap, no design direction — for a full month before a new product team was assigned, then another month while that team ramped up.

But the rebuild I want to describe isn't the external one — it's the internal one. Because the team that came out the other side of that experience was fundamentally different from the team that went into it.

When the layoffs happened, my team fractured along a predictable line. Some engineers went heads down — they focused on the work and didn't engage with the uncertainty. Some went the other direction — they started disengaging in meetings, updating resumes, asking questions about job security in every 1:1. And a small number did something different — they leaned in. They started proposing ideas, asking how they could help shape direction, treating the ambiguity as an opportunity rather than a threat.

That fracture gave me more information about my team than six months of normal sprints had. I could see clearly who had the initiative, the domain understanding, and the learning instinct to operate without scaffolding — and who needed more structure to perform well.

My first move was individual one on ones with every engineer within 48 hours. Not to manage their anxiety — to understand where each person actually was. I gave everyone the honest picture: engineering wasn't impacted, our skills weren't obsolete, here's the factual basis for why our team is safe. I didn't overpromise. I gave them the facts and let them draw their own conclusions.

My second move was deliberate role reshaping based on what the fractured team had revealed. I gave the engineers who had leaned in more ownership — not just tickets, but outcomes. I asked them to lead the AI capability building initiative during the ambiguous period. I asked them to define what we should be building, not just how. That stretch gave me signal on who was ready for the next level — and it gave those engineers the visibility and ownership that retained them through the uncertainty.

For the engineers who had gone quiet, I used the 1:1s differently. Less agenda, more conversation. I was trying to understand whether the disengagement was fear — which is manageable — or fundamental misalignment with the kind of environment we were building — which is a different conversation. In most cases it was fear, and transparency resolved it. In one case it became a longer conversation about whether this team was the right fit for that engineer's working style.

When the new product team arrived, I had already been meeting with the incoming PMs individually to understand their directional thinking. I'd started building toward their vision before requirements were finalized. When final specs landed, we already had a working solution close to target. Build time dropped from four months to three. Conversion improved 10%, NPS improved 15%.

Zero attrition through the entire period. Every engineer who was on the team when the layoffs happened was still on the team when we launched.

What I learned about reshaping a team is that the best moments to do it are the ones that feel like the worst moments — periods of disruption and uncertainty where normal patterns break down and you can see who people actually are when the scaffolding is gone. I didn't manufacture that disruption. But I was deliberate about what I did with the information it gave me. That's the difference between a leader who survives a crisis and one who uses it."

Follow-up: What do you do with an engineer who fundamentally doesn't fit the team's operating model?

"I have a direct conversation that separates the person from the fit question. Something like: 'I've noticed that the way we operate — high autonomy, ambiguous requirements, self-directed work — seems to be creating friction for you specifically. I want to understand whether that's a temporary adjustment challenge or whether a different kind of environment might actually be a better fit for how you work best.' That framing removes the blame and opens an honest conversation. Sometimes the engineer realizes they want more structure and we find a role or team that provides it. Sometimes they realize they want to grow into the ambiguity and we build a plan for that. What I don't do is pretend the misalignment isn't real — because leaving it unaddressed is worse for the engineer than naming it."

How do you build psychological safety on your team?

My definition of psychological safety is specific — it's the condition where engineers believe that raising a flag, admitting uncertainty, or disagreeing with a decision will not be held against them. It's not about comfort. It's about the absence of a specific fear: the fear that honesty has a cost.

The reason I'm precise about the definition is that psychological safety is easy to perform and hard to build. You can run retrospectives and call it psychological safety. You can say 'no stupid questions' and call it psychological safety. Neither of those things creates it. What creates it is behavior — specifically, what happens the first time someone takes a risk and is honest about something uncomfortable.

The clearest example I have is the PR review bottleneck on my team. A senior engineer with genuinely deep technical expertise was leaving extensive comments on PRs — mostly around style: naming conventions, indentation. PRs were getting stuck. Engineers were hesitating before opening them. After reviewing two sprints and noticing the velocity drop, I went into 1:1s to understand what was happening. What I found was that engineers weren't just frustrated — they were self-censoring. They were making decisions about what code to write based on what they thought would survive review, rather than what was actually right. That's the opposite of psychological safety.

I did three things. First, I had a direct conversation with the senior engineer privately. I kept all feedback anonymous to protect the people who had trusted me with it, but I was clear about the business impact — his review style was slowing our ability to ship and demoralizing the team. He pushed back initially. I stayed patient and kept explaining the impact until he became genuinely open to something different. That conversation itself was a psychological safety moment — I was modeling that hard feedback can be delivered and received without the relationship breaking.

Second, we co-created a solution together rather than me imposing one. We built a PR tagging system — Essential Fixes, Suggestions, and Nit Picks — so engineers had clarity on what actually blocked a merge. Co-creation was deliberate. When the senior engineer had ownership of the solution, he had investment in making it work.

Third, I introduced Husky to automate ESLint and indentation checks at commit — removing the most common style friction before it ever reached a PR. The structural fix addressed the symptom so the cultural fix could address the cause.

The results were immediate. PR merge time dropped to within 24 hours. Velocity increased 30%. But the result I cared most about was something harder to measure — engineers stopped hesitating before opening PRs. The moment I knew psychological safety had actually been restored was during a retrospective when another senior engineer spontaneously called out the improvements without being prompted. That kind of organic recognition only happens when the team genuinely feels the change, not just when they've been told about it.

What I've learned about building psychological safety is that it's established in moments, not initiatives. Every time someone raises a concern and I respond with curiosity instead of defensiveness, I'm building it. Every time I admit I don't know something in front of the team, I'm building it. Every time someone flags a risk early and I thank them rather than asking why they didn't catch it sooner, I'm building it. The team is always watching how I respond to honesty — and they calibrate their own honesty to match what they observe."

Follow-up: How do you build psychological safety with a new team where it doesn't exist yet?

"I start with myself. In the first few weeks I deliberately model the behaviors I want to see — admitting uncertainty in meetings, asking questions I don't know the answer to in front of the team, flagging my own mistakes before anyone else notices them. I also set explicit norms early: in retrospectives I make it clear that the goal is to surface problems, not assign blame, and I reinforce that by responding to every flagged problem with curiosity rather than analysis of who caused it. The fastest way to establish psychological safety with a new team is to create one high-visibility moment where someone takes a risk — flags something uncomfortable, disagrees with a decision, admits a mistake — and the outcome for them is positive. That first moment sets the expectation for every moment that follows."

How do you develop ICs toward Senior or Staff levels?

"My development philosophy for ICs is built on three mechanisms — structured knowledge building, visible ownership, and deliberate stretch. Each one addresses a different gap between where someone is and where they need to be.

The most complete example I have is an engineer I inherited when I transitioned into an EM role at Ancestry. We had been peers. In our first 1:1 he told me he expected a promotion soon. In reality he was trending toward a PIP. The gap between his self-perception and his actual impact was the largest I've encountered as an EM.

The first thing I did was a honest diagnostic conversation. Not to demoralize him — to understand the root cause before prescribing a solution. What I found was that the issue wasn't his coding ability. It was a lack of domain and architectural context. He didn't understand how our complex data systems were interconnected. Because of that gap he couldn't contribute to high-level design conversations, couldn't challenge product requirements, and couldn't see the downstream impact of his technical decisions. He was executing tickets accurately but operating below the surface of the system.

So I built a 90-day knowledge sprint specifically for him. Beyond our regular 1:1s I dedicated fifteen minutes each week to a technical deep dive. I'd explain a piece of the architecture and then give him a scavenger hunt task — not 'read this documentation' but 'find the specific data flow between these two systems and identify where the bottleneck is.' The goal was to build the instinct to go find answers independently, not just to transfer knowledge. At thirty days he started asking the right questions in grooming sessions. At ninety days he led a cross-team architectural discussion he would have avoided six months earlier.

That's the structured knowledge building mechanism. But knowledge alone doesn't get someone to Staff.

The second mechanism is visible ownership. I use our protected Friday learning time — no-meeting Fridays where engineers explore new technologies and ideas — specifically as a visibility platform. Engineers demo what they build. That demo structure does two things simultaneously: it creates accountability that produces real output, and it gives engineers practice presenting technical work to an audience. Both are Staff-level skills. The engineer I described earlier eventually demoed AI work during Friday sessions that directly influenced our product roadmap. That visibility accelerated his promotion case in ways that private 1:1 development never could have.

The third mechanism is deliberate stretch through ownership transfer. When I have competing priorities and need to distribute my attention, I don't give engineers tasks — I give them outcomes and let them figure out the path. I tell them explicitly: I'm trusting you with this outcome, not this ticket. The difference is significant. Owning a ticket requires execution. Owning an outcome requires judgment, communication, and the ability to navigate ambiguity — which are exactly the skills that separate Senior from Staff.

Over two years that engineer grew from near-PIP to Staff Engineer. Today he's one of the strongest mentors on the team — specifically helping new hires navigate the same domain complexity he once struggled with. The person who needed the most development became the person doing the most developing.

What I've learned about IC development is that the gap between Senior and Staff is almost never technical. It's always about scope of thinking — can this person see the system, not just the component? Can they influence without authority? Can they operate in ambiguity without needing direction? My development work is always aimed at expanding that scope — through knowledge, visibility, and ownership. The technical skills are already there. My job is to build the context and confidence that lets those skills operate at a higher level."

Follow-up: How do you handle an IC who wants to grow but isn't showing the behaviors that would justify promotion?

"I have a very direct conversation where I separate desire from readiness — and I make the gap concrete, not abstract. I don't say 'you're not ready yet.' I say 'here are the three specific things I need to see consistently before I can make your promotion case, here's what each one looks like in practice, and here's how we're going to create opportunities for you to demonstrate them.' The goal is to make the path to promotion feel actionable, not arbitrary. The worst thing I can do is leave someone in ambiguity about what they need to do — because ambiguity breeds resentment and eventually attrition. Clarity, even when it's hard to hear, is always the more respectful choice."

Tell me about a time you made a bad hire. How did you handle it?

"The bad hire I think about most was a senior engineer who interviewed exceptionally well — technically strong, articulate, great at explaining their thinking under pressure. On paper they were exactly what we needed. The technical bar was met clearly. The culture signal felt positive in the room.

What I missed was the operating model. Their previous success had come from a highly structured environment where requirements were handed down clearly, decisions were made above them, and the definition of good work was executing accurately to spec. When they joined our team — where engineers were expected to push back on product requirements, shape their own work, and make judgment calls with incomplete information — they struggled significantly.

It wasn't dramatic. They didn't fail publicly or create conflict. They just… waited. They delivered exactly what was asked but never challenged whether it was the right thing to build. In grooming sessions they went quiet when the team was debating requirements. When product came with something that didn't make sense technically, they accepted it and built it rather than flagging the issue. In a team where psychological safety meant speaking up, their silence started to read as disengagement to peers — and over time it was.

The values weren't wrong. They were hardworking, reliable, and genuinely well-intentioned. The operating model was wrong — and operating model mismatches are much harder to fix than skill gaps because they're not about what someone knows, they're about how someone fundamentally thinks their job works.

When I recognized what was happening — about three months in — I had a direct conversation. I named what I was observing specifically: 'I've noticed that in situations where the team is debating a requirement or pushing back on a product decision, you tend to go quiet. I want to understand whether that's intentional or whether there's something about those conversations that feels uncomfortable.' That framing gave them room to be honest without feeling attacked.

What came out of that conversation was genuinely useful — they explained that in their previous environment, pushing back on product was actively discouraged. Engineers who challenged requirements were seen as obstacles. They'd spent years learning to suppress exactly the instinct I was trying to develop in them. That context reframed the situation for me. This wasn't a values problem — it was a conditioned behavior that had been reinforced for years.

So I built a structured plan to recondition it. In grooming sessions I started explicitly inviting their perspective on requirements before the group debate started — creating a lower-stakes moment for them to practice voicing opinions. I paired them with engineers who modeled the pushback behavior well. And I debriefed with them after meetings where they'd had the opportunity to speak up — not to critique them but to ask what they'd been thinking and why they hadn't said it.

Progress was real but slow. They did develop the instinct over time — but never fully to the level the team operated at. Eventually they moved to a role at another company with a more structured

environment, and from what I heard they thrived there immediately. Which was actually the right outcome — just not one I'd engineered deliberately.

What I changed in my hiring process afterward was specific. I added one question to every senior engineer loop: 'Tell me about a time you disagreed with a product requirement or a decision made above you. What did you do?' Candidates from highly structured environments almost always describe escalating through proper channels or accepting the decision. Candidates who thrive with autonomy describe challenging it, proposing an alternative, or running an experiment. That one question now tells me more about operating model fit than any technical assessment. I also added a follow-up: 'How did you decide what to push back on versus what to accept?' That follow-up reveals their risk philosophy — and risk philosophy is what you're actually hiring for at the senior level.

The most important thing I learned is that bad hires are almost never about skills — they're almost always about context I failed to assess. My job in the interview isn't to validate that someone is good. It's to understand specifically what conditions they need to be good — and to be honest about whether we can provide those conditions."

Follow-up: How do you decide when to keep investing in someone vs when to accept the mismatch?

"My decision point is whether the gap is behavioral or structural. Behavioral gaps — specific habits or patterns that someone can change with awareness and practice — I invest in. The conditioned suppression of pushback is behavioral. It took time but it was movable. Structural gaps — fundamental misalignments in how someone thinks their job works, what they value, what they want from work — I address more directly and more quickly. The honest conversation about fit is kinder than a prolonged period of mutual disappointment. The signal I watch for is whether the person themselves recognizes the gap and wants to close it. If they're working against the change rather than toward it, that tells me the gap is structural and the kindest thing I can do is help them find a better fit — for both of us."

How do you keep your team motivated during periods of uncertainty or org change?

"My approach to team motivation during uncertainty runs on two tracks simultaneously — addressing the fear directly and creating meaningful work that pulls people forward rather than just managing their anxiety about what's behind them.

The most complete example I have combines two experiences that happened in sequence at Ancestry.

The first was when our entire product and UX org was laid off overnight. My team of twelve went from daily collaboration to complete silence. People were updating resumes, going quiet in meetings, asking about job security in every 1:1. The fear was real and legitimate.

My first move was individual one on ones with every engineer within 48 hours — not to give them answers I didn't have, but to understand where each person actually was. Then I gave them the honest picture: engineering wasn't impacted, our skills weren't obsolete, here's the specific factual basis for why our team is safe. I didn't manage their anxiety with reassurance — I displaced it with information. There's a difference. Reassurance says 'it'll be fine.' Information says 'here's what I know and here's what I don't know and here's what I'm doing about the second part.' People can handle uncertainty. What they can't handle is feeling like they're being managed rather than trusted.

The second move was refusing to let the team drift. I made the decision immediately to keep everyone productive — continuing our existing sprint backlog under the assumption the new product team would evaluate alignment, and simultaneously directing the team toward AI capability building. I framed it explicitly: we don't have product direction right now, so let's use this time to build something that will make us more valuable regardless of which direction we go. That reframe transformed a period of paralysis into a period of genuine growth.

The second experience that shaped my approach was noticing — four months into an intensive delivery project — that the team's creative energy had disappeared. Discussions had become repetitive cycles of the same solutions. Nobody was proposing new approaches. People were executing without curiosity. I realized I'd fallen into the same trap myself — I hadn't learned anything new in months.

So I introduced no-meeting Fridays. Protected time away from delivery pressure specifically for learning, exploration, and building things that excited people. I modeled it myself first — I used Fridays to learn LangChain, built a chatbot, and demoed it to the team. Engineers followed. The energy shift was immediate and hard to miss. Within weeks people were bringing genuinely creative approaches to the onboarding project, suggesting new functionality to product partners, and collaborating on what eventually became Ancestry University — a project the product team adopted as a feature concept.

The common thread across both experiences is this: motivation during uncertainty doesn't come from managing how people feel about the uncertainty. It comes from giving them something

meaningful to move toward. Fear looks backward — at what's been lost or what might go wrong. Meaningful work looks forward. My job in a crisis is to redirect attention forward as quickly as possible, through honest information and real work that matters.

The signal I watch most closely during periods of uncertainty is whether people are bringing ideas to meetings or just showing up. A team that's still generating ideas — even small ones, even half-formed ones — is a team that's still engaged. A team that goes quiet is a team that's already left mentally. I catch that signal early in 1:1s and address it before it spreads."

Follow-up: What do you do when you yourself don't know what's going to happen — when you're as uncertain as your team?

"I say so. Directly. The worst thing I can do is perform confidence I don't have — because engineers are smart and they can tell, and the moment they catch me performing, they stop trusting everything else I say. What I've learned is that leaders don't have to have the answers to be effective during uncertainty. They have to be honest about what they know, decisive about what they can control, and calm in a way that's grounded rather than performed. I tell my team: here's what I know, here's what I don't know, here's what I'm doing to find out, and here's what we're going to do in the meantime. That combination of honesty and agency is what keeps a team functional even when the future is genuinely unclear."

What's your philosophy on giving feedback and how do you make it stick?

"My feedback philosophy starts from one conviction — feedback only sticks when the person receiving it understands specifically what changed and why it matters to them personally. Generic feedback doesn't stick. Feedback delivered too late doesn't stick. Feedback delivered without a path forward doesn't stick.

I think about feedback across three dimensions: timing, specificity, and ownership.

On timing — I give feedback as close to the moment as possible. Not in the next 1:1, not in the next review cycle — as close to the observation as the relationship and context allow. The reason is simple: the further you get from the moment, the more abstract the feedback becomes. An engineer who got a PR comment wrong three months ago can't connect the feedback to the memory. An engineer who got it wrong this week can.

The clearest example of timing mattering was the PR review bottleneck on my team. A senior engineer's review style was slowing the team — extensive style comments, blocked merges, declining velocity. I didn't wait for a review cycle. I gathered signal from 1:1s across the team first, then had the direct conversation with him within the same two-week period I'd identified the problem. The proximity to the behavior was what made the conversation actionable rather than abstract.

On specificity — I never give feedback without a specific example that makes the pattern concrete. Not 'your communication could be clearer' but 'in yesterday's grooming session when you explained the hint generation issue, I noticed the product team looked confused and didn't ask follow-up questions — which usually means the explanation didn't land the way you intended.' Specific examples remove the defensiveness that abstract feedback triggers — because there's nothing to argue with. Either it happened or it didn't.

When the senior engineer pushed back on my feedback initially, I didn't retreat to the abstract. I stayed specific — these are the PRs that took longer than 24 hours to merge, these are the velocity numbers across two sprints, these are the engineers who told me in 1:1s they were hesitating before opening PRs. Specificity is what moved him from defensive to genuinely open.

On ownership — the most important moment in any feedback conversation is not when I deliver the observation. It's when I ask the person receiving it what they want to do about it. Co-creating the solution builds ownership of the change in a way that a prescribed solution never does. When the senior engineer and I designed the PR tagging system together — Essential Fixes, Suggestions, Nit Picks — he had investment in making it work because it was partly his solution. It stuck because it was his, not mine.

The structural change I've made that has the highest impact on feedback sticking is separating feedback conversations from performance conversations. If feedback only happens during reviews, people hear it through a compensation lens — which completely distorts the signal. I give ongoing feedback throughout the year specifically so that when review time comes, nothing in the

review is a surprise. The review becomes a summary of a conversation we've been having all year, not the conversation itself.

What I've learned about making feedback stick is that the goal isn't to deliver a message — it's to shift a behavior. Those are different success criteria. A message is delivered when someone hears it. A behavior shifts when someone understands it, owns it, and has a concrete path to change it. My job isn't done when I've said the thing. It's done when I can observe the change."

Follow-up: How do you give feedback to someone who is resistant to it?

"I stay specific and I stay patient — in that order. Resistance to feedback almost always comes from one of two places: either the person disagrees with the observation, or they agree with it but feel defensive about what it implies about them. I distinguish between these early. If they're disputing the observation, I go back to the specific example — not to win the argument but to anchor the conversation in what actually happened. If they're defensive about the implication, I separate the observation from the judgment. 'I'm not saying you're a bad engineer — I'm saying this specific behavior is having this specific impact, and I think you can change it.' The goal is to make the feedback feel like information rather than a verdict. Verdicts create defensiveness. Information creates problem-solving."